UNIVERSITÀ DI PARMA

Data Types in C

What's in a name? That which we call a rose,
By any other name would smell as sweet.

William Shakespeare, Romeo and Juliet,
Act II, Scene II

Summary

UNIVERSITÀ DI PARMA

- What is a “variable”?
- Data Type in C
- Numbers
  - Floating point data types
  - Integer data types
- Combining different types
  - Cast
- Visibility & Lifetime

Variables

UNIVERSITÀ DI PARMA

- Something that is able to "vary"
- Definition:
  - **Memory space** with a **name** where it is possible to **store data** during program execution
- Memory space ⟶ address & size
  - Automatically managed by high level programming languages

Terminology

- Content or value of a variable → right value or rvalue
- Memory address → left value or lvalue
- Size

```
int a = 173;
```

Name of a variabile

UNIVERSITÀ DI PARMA

- Rules
  - Only letters, digits, or underscore “_”
  - It must begin with a letter (or “_”)
  - Names are case sensitive
  - Reserved words can not be used

Name of a variable

UNIVERSITÀ DI PARMA

- Recommended style:
  - Select small caps for variable names
  - Use meaningful names
  - When the name is composed by multiple words use uppercase letters or underscore to emphasize that

```
int sum             = 173;
int students_number = 144;
int ClassId         = 10;
```

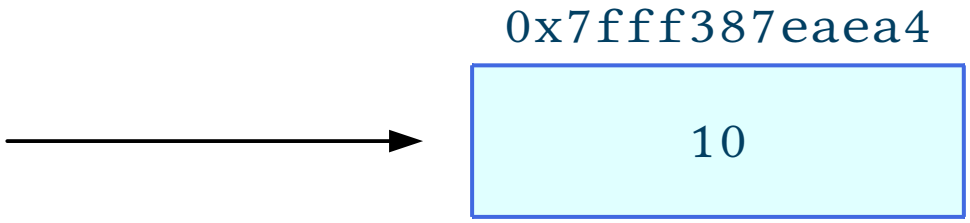
How to define a variable

UNIVERSITÀ DI PARMA

```
// variables are defined as
// <type> <variable_name>;

int a;
   // I am defining a “int” variable named ‘b’
   // in such a case the variable is NOT initialized
   // this means that ‘a’ contains a RANDOM number

int b=17;
   // I am defining AND initializing a “int” variable named ‘b’
```

Defining & initing a variable

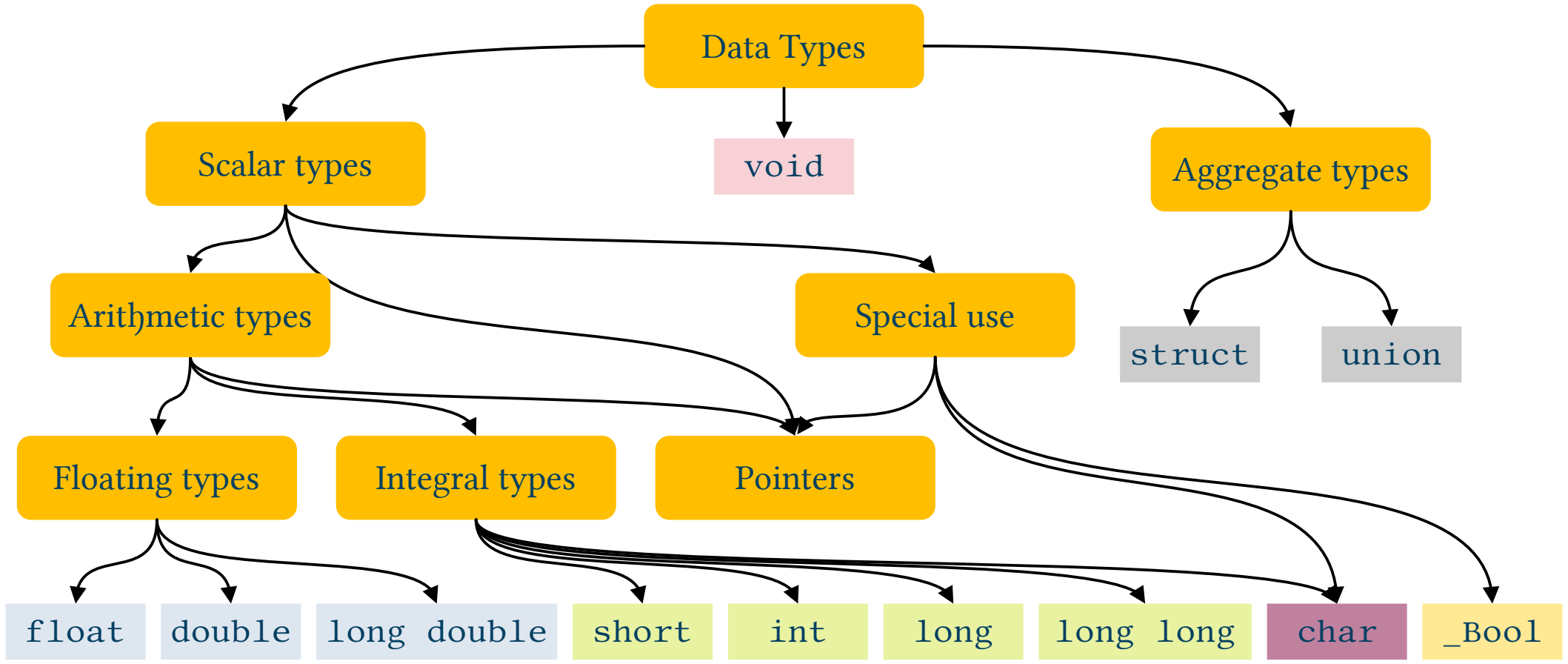

```
int a;

a = 10;
```



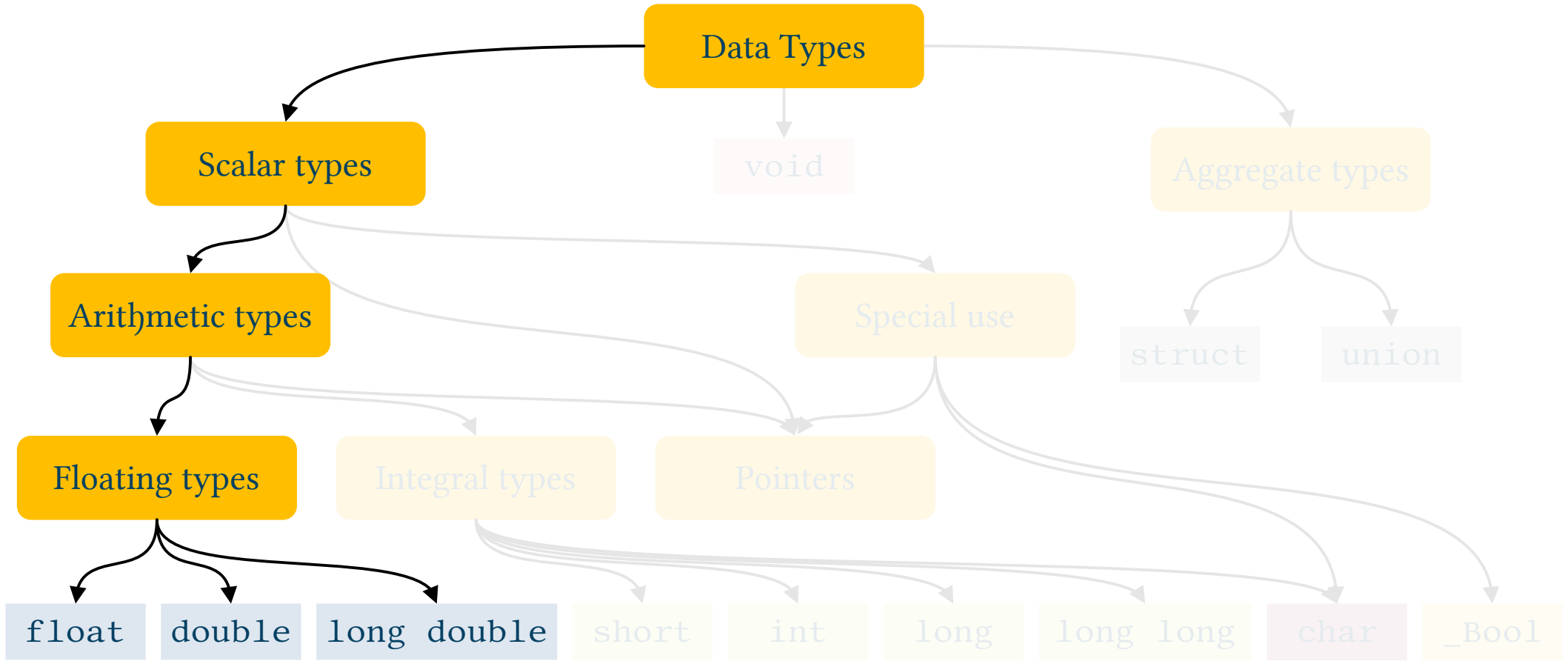
- Definition of a variable does not initialize it
  - Random value
  - Do not rely on it being Ø
- Always init variables before using them!

C99 Hierarchy of Data Types (simplified)

UNIVERSITÀ DI PARMA



Floating point types



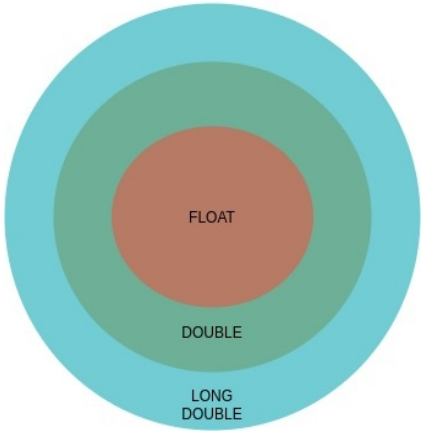
Floating point types

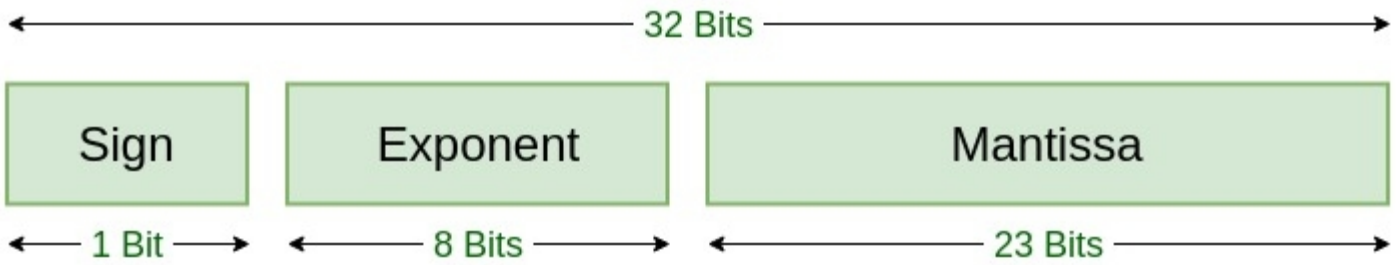
UNIVERSITÀ DI PARMA

- They can hold very large numbers or fractions
  - Positive or negative numbers
  - With or without the fractional part
  - Remember approximation
- Why we have approximation?
  - Limited size for the fractional part
  - Min & Max values limit

Floating point types

- 3 possibilities
  - float → single precision
  - double → “double” precision
  - long double → extended precision
- Main difference?
  - Number of bytes used to store the data

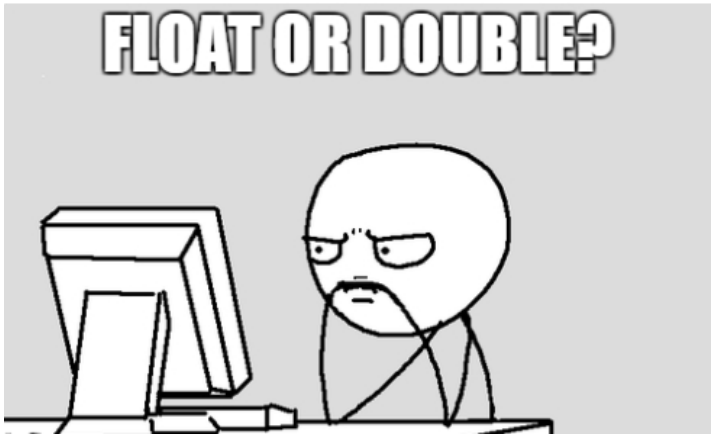




Float vs Double vs Long Double

UNIVERSITÀ DI PARMA

- Which type should I use?
- As always it is a compromise among:
  - Precision
  - Execution speed
  - Memory requirements



Floating Point Types

UNIVERSITÀ DI PARMA

- The C standard recommends the IEEE 754 format?
  - Actually not, but this is not important
- The C standard says anything about the size of different types?
  - No
- It is up to the compiler!
  - Often, there is not difference between float or double types

sizeof()

UNIVERSITÀ DI PARMA

- We have a tool to understand the size of a variable
- sizeof(<data type>) or sizeof(<variable name>)
  - It returns the number of bytes used to store that data
- A specific prefix ('z') for a correct format specifier
  - %zd C99
  - %ld C89 – use this one…

Floating Point Types

UNIVERSITÀ DI PARMA

- How to print or read them?
  - Formats
    - %f → decimal notation 123.456000
    - %e → scientific notation 1.234560e+02
    - %E → scientific notation 1.234560E+02
    - %g → shortest between %f and %e 123.456
  - Suffissi per tipo
    - “none” → float
    - l → double (only mandatory for input)
    - L → long double

Floating Point Types

UNIVERSITÀ DI PARMA

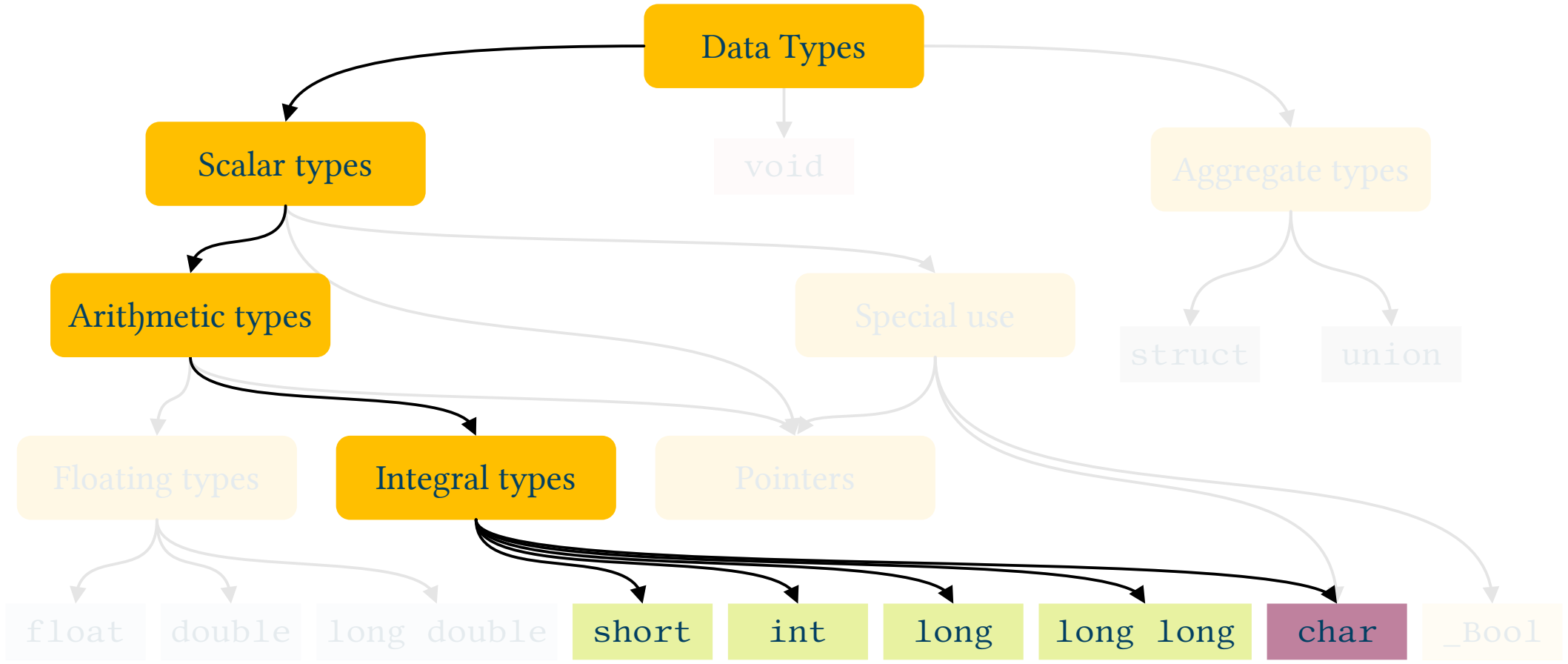
- Specific values
- ±inf ⟶ Infinite
  - Usually as a result of math operations
  - i.e. division by zero
- NaN ⟶ not a number
  - Probably some memory error
- Specific functions
  - isnan(), isinf()...

Tackling floating point precision issue

UNIVERSITÀ DI PARMA

- We saw many examples of “problems” with precision and floating point types. Best practices:
  - Use Appropriate Data Types
    - **Double Precision:** use double instead of float when a high degree of accuracy is required
    - **Fixed-point Arithmetic:** where exact decimal representation is crucial, consider using fixed-point arithmetic (i.e. finance)
  - **Avoid Equality Comparisons**
    - Instead of direct comparison (==), check if numbers are "close enough":

Integer Types



Integer Types

UNIVERSITÀ DI PARMA

- 4+1 types:
  - char → seldom used for “numbers”
  - short int
  - int → probably the most used data type
  - long int
  - long long int
- The difference is, again, the size of memory used to store them
  - Given *n* bits I can store non negative numbers in the $0 - 2^n-1$ interval

Integer Types

UNIVERSITÀ DI PARMA

- All integer types can deal with negative numbers ("signed" types)
  - The range of numbers I can store is then not $[0, 2^n-1]$ but $[-2^{n-1}, 2^{n-1}-1]$
- By default all signed types but
  - char – compiler dependent
- When I use them I can change the default behavior
  - unsigned
  - (signed)
- This brings the number of possible integer types to 10!

Integer Types (typical example for 64 bit)

UNIVERSITÀ DI PARMA

Type	Byte	Smallest Value	Largest Value
signed char	1	-128	127
unsigned char	1	0	255
short	2	-32.768	32.767
unsigned short	2	0	65.535
int	4	-2.147.483.648	2.147.483.647
unsigned int	4	0	4.294.967.295
long	8	-9.223.372.036.854.775.808	9.223.372.036.854.775.807
unsigned long	8	0	18.446.744.073.709.551.615
long long	8	-9.223.372.036.854.775.808	9.223.372.036.854.775.807
unsigned long long	8	0	18.446.744.073.709.551.615

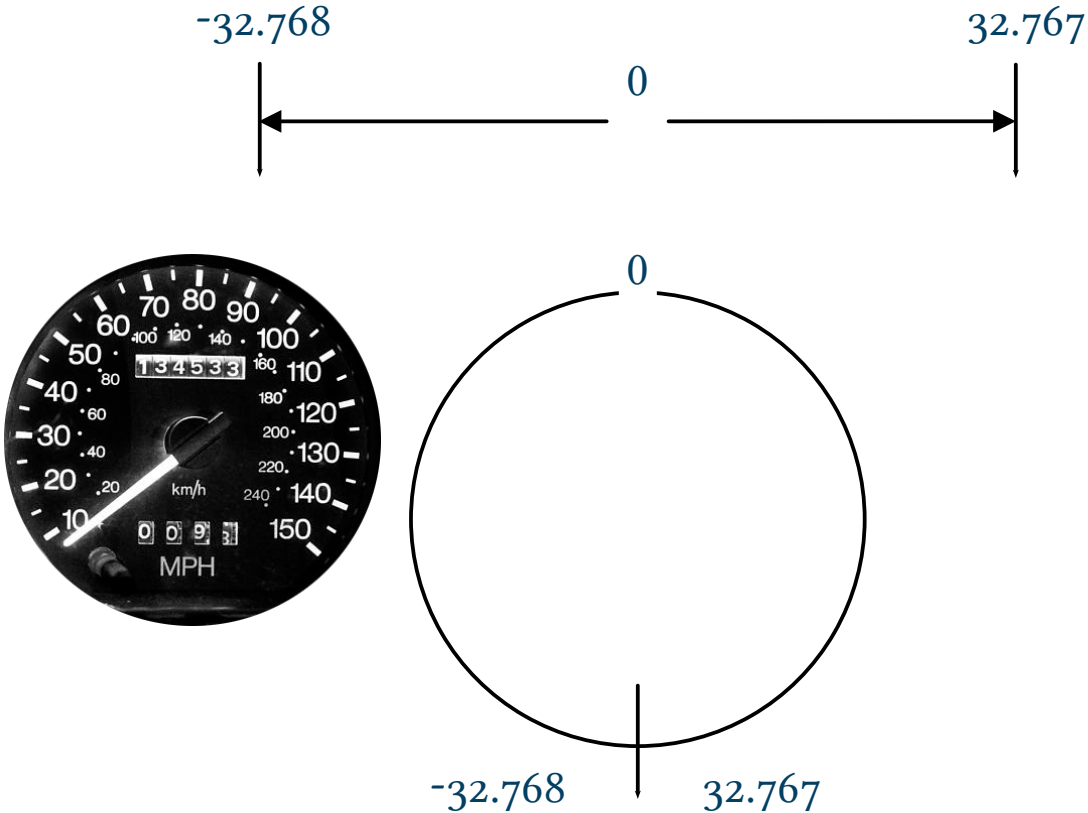
Integer Types

UNIVERSITÀ DI PARMA

- How we can read or print them?
  - Formats
    - %d → decimal notation -45
    - %x → hexadecimal notation ffffffd3
    - %X → hexadecimal notation FFFFFFD3
    - %u → decimal notation for unsigned types 4294967251
  - Suffixes
    - “none” → int
    - hh → char
    - h → short
    - l → long
    - ll → long long

Overflow

- When an operation produces a result too big (or too small) for the given type
- No execution error!
  - Usually clamping

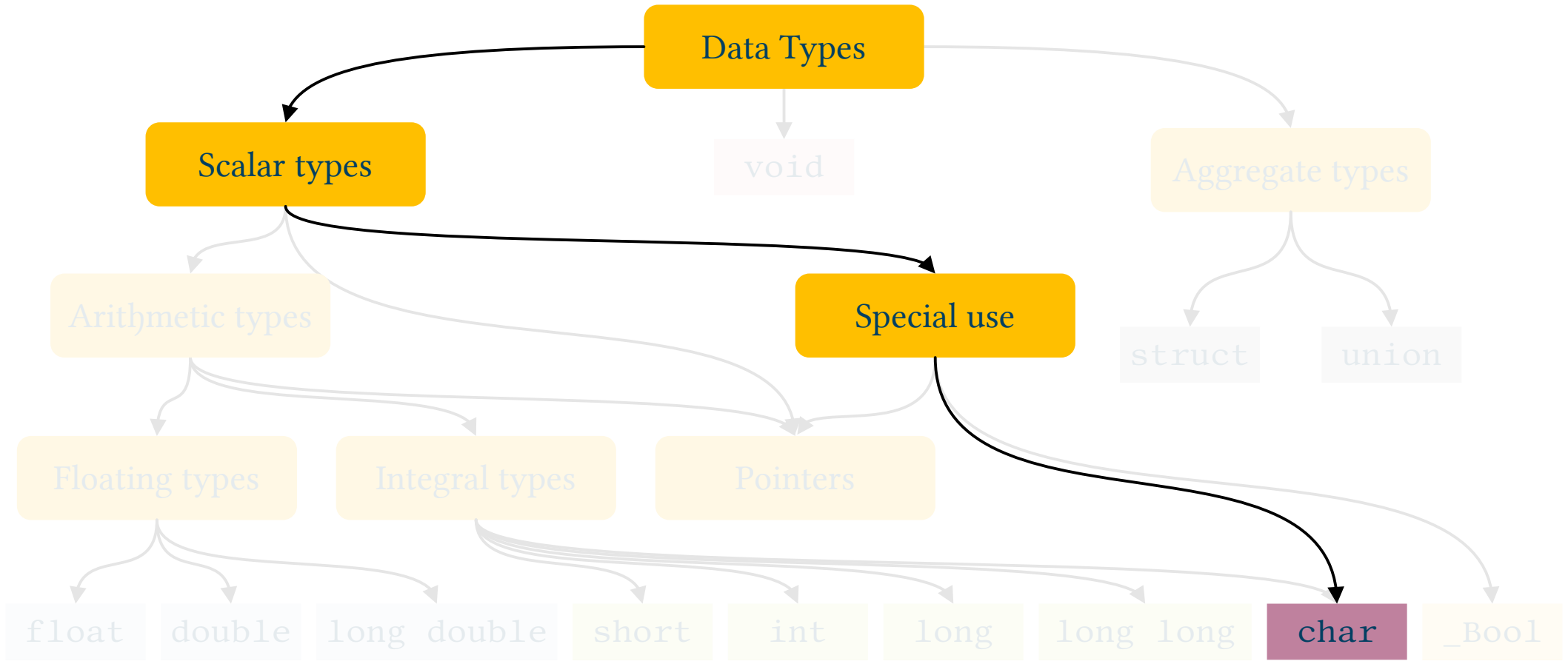


Overflow

UNIVERSITÀ DI PARMA

- In the speed control a float was cast into an unsigned 16 bit integer
  - unsigned short
  - 0 – 65.535 range…
- ~370.000.000 USD

Char Type


Data Types
Scalar types
void
Aggregate types
Arithmetic types
Special use
struct
union
Floating types
Integral types
Pointers
float
double
long double
short
int
long
long long
char
_Bool

Char Type

UNIVERSITÀ DI PARMA

- It is used to store numbers!
  - 0–255
  - -128–127
- But typically only used to store ASCII codes
- Format specifier
  - %c
- Constants:
  - Direct use of magic numb 65
  - ~~Use of correspondent symbol 'A'~~

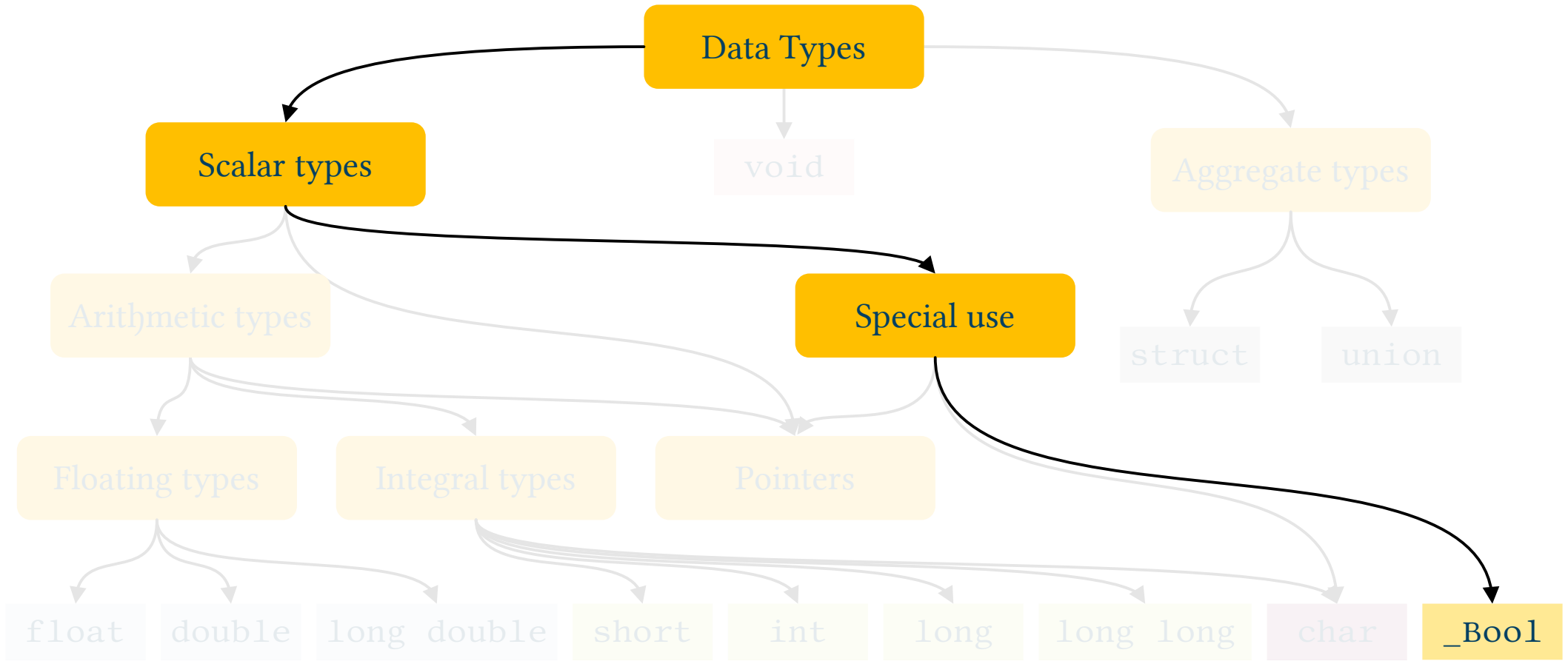
The char type & scanf()

UNIVERSITÀ DI PARMA

- Reading the char and derived types using scanf() can be a little tricky
- When I type on the keyboard all characters are fed as input to the program
  - All means all. Namely, also the enter key…
  - All those "key presses" are initially stored in a input buffer


```
scanf("%c", &x); // I also read spaces and newlines


scanf(" %c", &x); // Ignore leading spaces or newlines
```

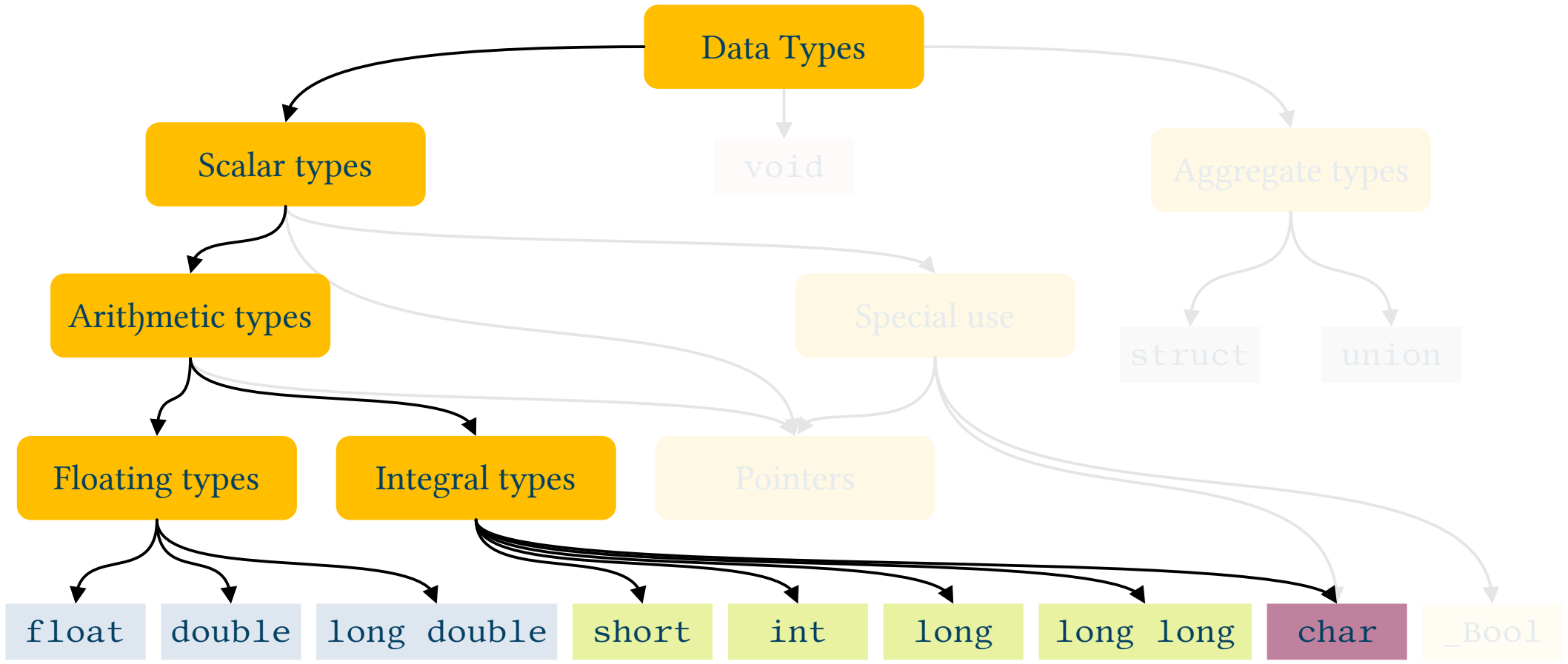
Boolean Type



The _Bool Type

- C99 (and later) specific
- A integer data type with the aim to only store “true” or “false”
  - 0 ⟶ false
  - 1 ⟶ true
- This type can then features 1 or 0 values only

“Arithmetic” issues



Combining different data types

UNIVERSITÀ DI PARMA

- 3 floating point types and 8–10 integer ones
- Can I combine them in operations?
  - Yes
- What is the C behavior when I mix different kind of data types?
- Conversions
  - Assignment
  - Implicit or automatic conversions

Assignment conversions

UNIVERSITÀ DI PARMA

- When I operate an assignement (=)
  - The right operand (rvalue) is "casted" to the type I have on the left operand (lvalue)
- Issues
  - I can lost information i.e.: float → int
  - Wrong result i.e.: long long → short

Implicit or automatic casting

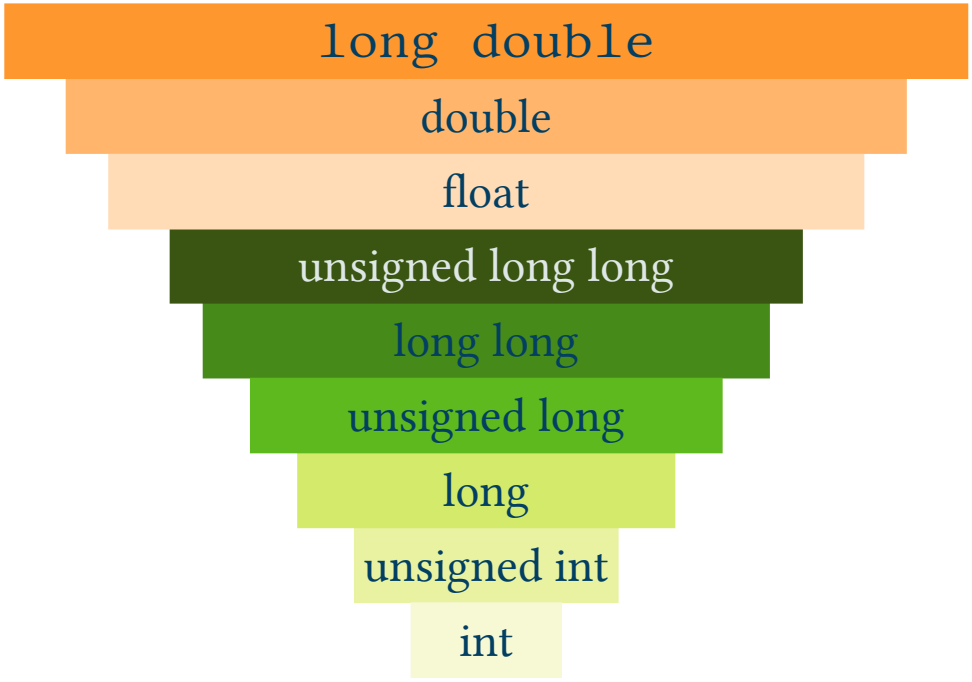
UNIVERSITÀ DI PARMA

- It occurs when we combine different types in expressions
- Binary operands needs same type operators
  - Arithmetic: *, /, %, +, -; Comparisons: <, >, <=, >=, ==, !=; Bitwise: &, ^, |,
- i.e.: a+b when a is an int and b is a float which kind of type gives as a result?
  - Spoiler: float
- Conversion rules
- High probability of bugs!

Implicit cast rules

UNIVERSITÀ DI PARMA

- Some types are converted to int or unsigned int
  - Even when we have same type operands
  - _Bool, char e short → int
    - Or unsigned int
- After this → conversion pyramid



Explicit cast

UNIVERSITÀ DI PARMA

- I select the conversion
  - This lead to avoid errors
- Syntax:

```
(type) expression
```

Variable scope

UNIVERSITÀ DI PARMA

- Can I use a variable everywhere in my code?
  - No!
- Visibility field or scope
  - Part of the code where a variable can be used
- 3 cases:
  - Program (global variables)
  - File
  - Block (local variables)

Global Variables

- Defined outside of every code block
  - {}
- I can modify them everywhere
  - Data are NOT protected
  - Debug becomes more difficult
  - Uncertain
- Your final score will be, FOR SURE, affected
- One single benefit ⟶ initialization to 0

Local Variables

UNIVERSITÀ DI PARMA

- Defined
  - Inside a code block i.e. inside {}
  - In a for(;;)
  - As function arguments
- I can use them only inside the specif block where they are defined
- Automatic duration
  - Created when the execution flow meets that code block
  - Destroyed when that code block is over

Global vs Local

UNIVERSITÀ DI PARMA

- Local Variables
  - Let code blocks or functions to be self-sufficient
  - Easier to manage or to understand the code
    - Modularity
  - Less portability issues
- Global Variables
  - I have to track all the code parts where they are used
  - Require to coordinate team work
  - Memory is always used
  - Naming issues

Variables Duration

- A variable is not always forever
- Static lifetime
  - Global Variables
    - Memory reserved when the execution begins and never freed
- Automatic lifetime
  - Local variables
    - Local variables are created when code execution reaches that block
    - And destroyed when the execution is over
    - Can I change this behavior? Yes → `static`

Constant values

UNIVERSITÀ DI PARMA

- We can need to use constant values
- Quick and dirt approach:
  - Put in my code a lot of "magic numbers"
  - This lead to a code really difficult to modify or maintain
- Good approach:
  - Define constant
  - Parametric code, very easy to modify or improve

e π μ

How to define a constant value in C

UNIVERSITÀ DI PARMA

- Three different approaches
- #define
  - i.e.: `#define N_MAX_STUDENTS (100)`
- const prefix
  - I define and init a variable whose initial value can not be modified
  - In such a case the use of a global variable makes a sense
  - i.e.: `const int N_MAX_STUDENTS=100;`

UNIVERSITÀ DI PARMA

Data Types in C

What's in a name? That which we call a rose,
By any other name would smell as sweet.

William Shakespeare, Romeo and Juliet,
Act II, Scene II

Summary

UNIVERSITÀ DI PARMA

- What is a “variable”?
- Data Type in C
- Numbers
  - Floating point data types
  - Integer data types
- Combining different types
  - Cast
- Visibility & Lifetime

Variables

UNIVERSITÀ DI PARMA

- Something that is able to "vary"
- Definition:
  - **Memory space** with a **name** where it is possible to **store data** during program execution
- Memory space ⟶ address & size
  - Automatically managed by high level programming languages

Terminology

UNIVERSITÀ DI PARMA

- Content or value of a variable → right value or rvalue
- Memory address → left value or lvalue
- Size

int a = 173;

Name of a variabile

UNIVERSITÀ DI PARMA

- Rules
  - Only letters, digits, or underscore "_"
  - It must begin with a letter (or "_")
  - Names are case sensitive
  - Reserved words can not be used

Name of a variable

UNIVERSITÀ DI PARMA

- Recommended style:
  - Select small caps for variable names
  - Use meaningful names
  - When the name is composed by multiple words use uppercase letters or underscore to emphasize that

```
int sum             = 173;
int students_number = 144;
int ClassId         = 10;
```

How to define a variable

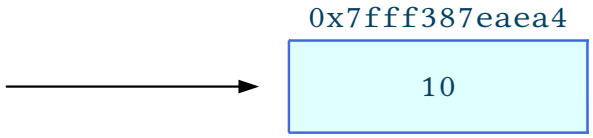
UNIVERSITÀ DI PARMA

```
// variables are defined as
// <type> <variable_name>;

int a;
   // I am defining a “int” variable named ‘b’
   // in such a case the variable is NOT initialized
   // this means that ‘a’ contains a RANDOM number

int b=17;
   // I am defining AND initializing a “int” variable named ‘b’
```

Defining & initing a variable

UNIVERSITÀ DI PARMA

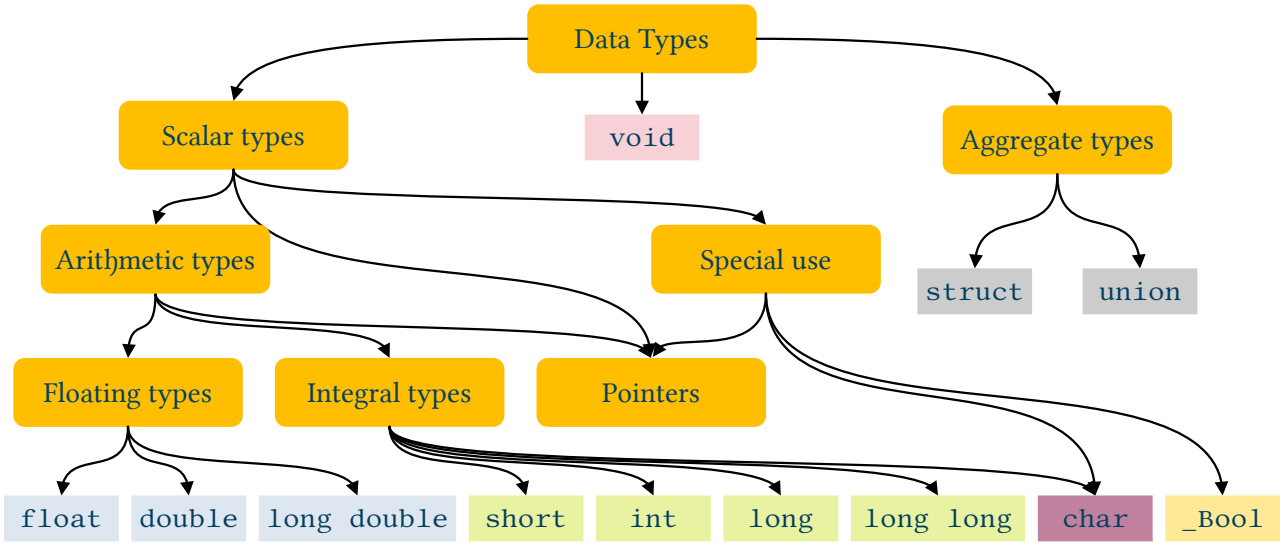
```
int a;

a = 10;
```



- Definition of a variable does not initialize it
  - Random value
  - Do not rely on it being Ø
- Always init variables before using them!

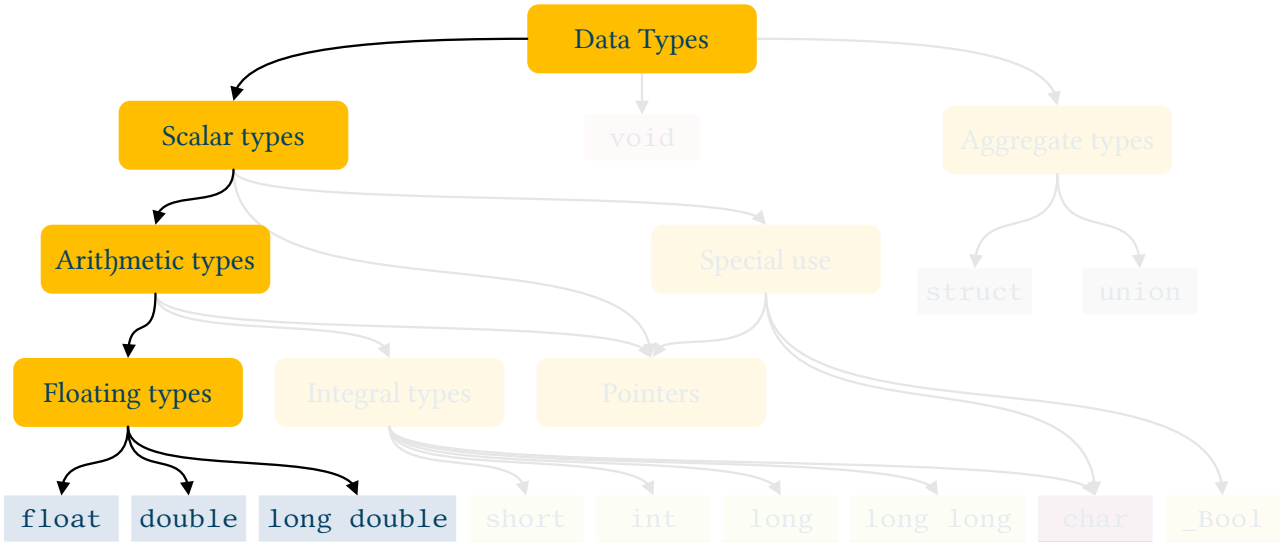
C99 Hierarchy of Data Types (simplified)

UNIVERSITÀ DI PARMA



Floating point types

UNIVERSITÀ DI PARMA



Floating point types

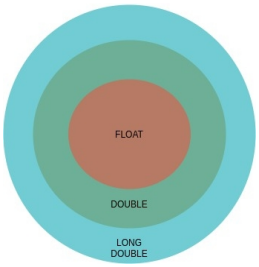
UNIVERSITÀ DI PARMA

- They can hold very large numbers or fractions
  - Positive or negative numbers
  - With or without the fractional part
  - Remember approximation
- Why we have approximation?
  - Limited size for the fractional part
  - Min & Max values limit

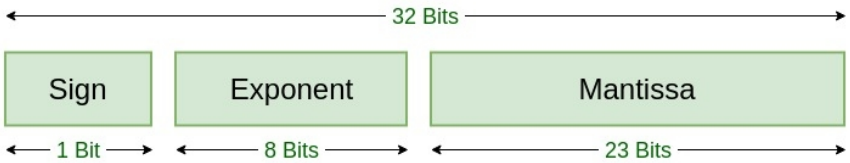
Floating point types

UNIVERSITÀ DI PARMA

- 3 possibilities
  - float → single precision
  - double → "double" precision
  - long double → extended precision



- Main difference?
  - Number of bytes used to store the data

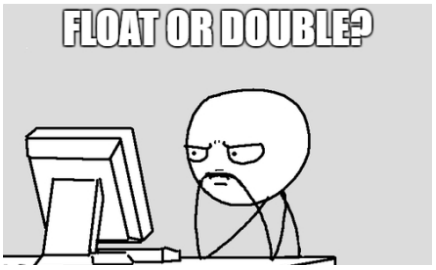


Float → 23 bit di precisione
Double → 52 bit

Float vs Double vs Long Double

UNIVERSITÀ DI PARMA

- Which type should I use?
- As always it is a compromise among:
  - Precision
  - Execution speed
  - Memory requirements



Floating Point Types

UNIVERSITÀ DI PARMA

- The C standard recommends the IEEE 754 format?
  - Actually not, but this is not important
- The C standard says anything about the size of different types?
  - No
- It is up to the compiler!
  - Often, there is not difference between float or double types

sizeof()

UNIVERSITÀ DI PARMA

- We have a tool to understand the size of a variable
- sizeof(<data type>) or sizeof(<variable name>)
  - It returns the number of bytes used to store that data
- A specific prefix ('z') for a correct format specifier
  - %zd C99
  - %ld C89 – use this one…

Floating Point Types

UNIVERSITÀ DI PARMA

- How to print or read them?
  - Formats
    - %f → decimal notation 123.456000
    - %e → scientific notation 1.234560e+02
    - %E → scientific notation 1.234560E+02
    - %g → shortest between %f and %e 123.456
  - Suffissi per tipo
    - "none" → float
    - l → double (only mandatory for input)
    - L → long double

Floating Point Types

UNIVERSITÀ DI PARMA

- Specific values
- ±inf ⟶ Infinite
  - Usually as a result of math operations
  - i.e. division by zero
- NaN ⟶ not a number
  - Probably some memory error
- Specific functions
  - isnan(), isinf()…

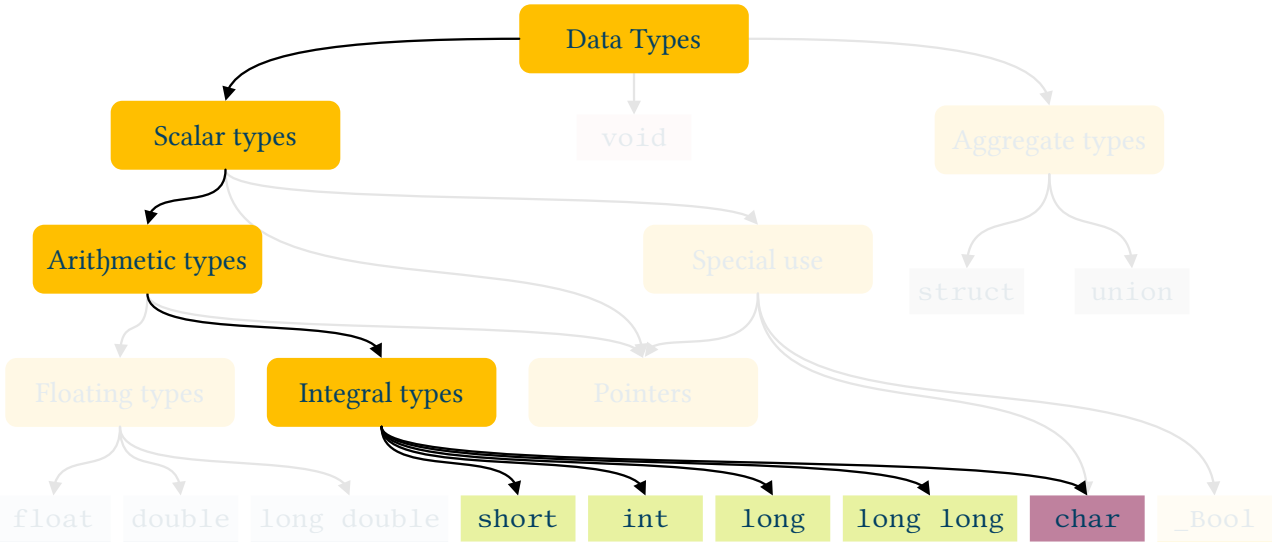
Tackling floating point precision issue

UNIVERSITÀ DI PARMA

- We saw many examples of “problems” with precision and floating point types. Best practices:
  - Use Appropriate Data Types
    - **Double Precision:** use double instead of float when a high degree of accuracy is required
    - **Fixed-point Arithmetic:** where exact decimal representation is crucial, consider using fixed-point arithmetic (i.e. finance)
  - **Avoid Equality Comparisons**
    - Instead of direct comparison (==), check if numbers are "close enough":

Integer Types

UNIVERSITÀ DI PARMA



Integer Types

UNIVERSITÀ DI PARMA

- 4+1 types:
  - char → seldom used for "numbers"
  - short int
  - int → probably the most used data type
  - long int
  - long long int
- The difference is, again, the size of memory used to store them
  - Given *n* bits I can store non negative numbers in the $0 - 2^n-1$ interval

Integer Types

UNIVERSITÀ DI PARMA

- All integer types can deal with negative numbers ("signed" types)
  - The range of numbers I can store is then not $[0, 2^n-1]$ but $[-2^{n-1}, 2^{n-1}-1]$
- By default all signed types but
  - char – compiler dependent
- When I use them I can change the default behavior
  - unsigned
  - (signed)
- This brings the number of possible integer types to 10!

Integer Types (typical example for 64 bit)

UNIVERSITÀ DI PARMA

Type	Byte	Smallest Value	Largest Value
signed char	1	-128	127
unsigned char	1	0	255
short	2	-32.768	32.767
unsigned short	2	0	65.535
int	4	-2.147.483.648	2.147.483.647
unsigned int	4	0	4.294.967.295
long	8	-9.223.372.036.854.775.808	9.223.372.036.854.775.807
unsigned long	8	0	18.446.744.073.709.551.615
long long	8	-9.223.372.036.854.775.808	9.223.372.036.854.775.807
unsigned long long	8	0	18.446.744.073.709.551.615

Integer Types

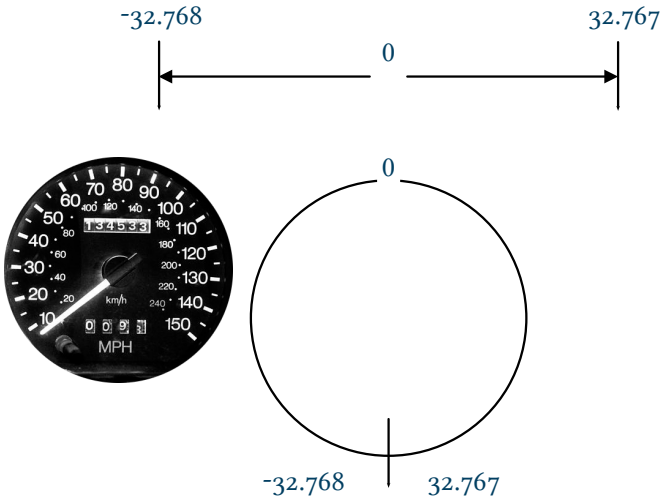
UNIVERSITÀ DI PARMA

- How we can read or print them?
  - Formats
    - %d → decimal notation -45
    - %x → hexadecimal notation fffffd3
    - %X → hexadecimal notation FFFFFFD3
    - %u → decimal notation for unsigned types 4294967251
  - Suffixes
    - “none” → int
    - hh → char
    - h → short
    - l → long
    - ll → long long

Overflow

UNIVERSITÀ DI PARMA

- When an operation produces a result too big (or too small) for the given type
- No execution error!
  - Usually clamping



Overflow

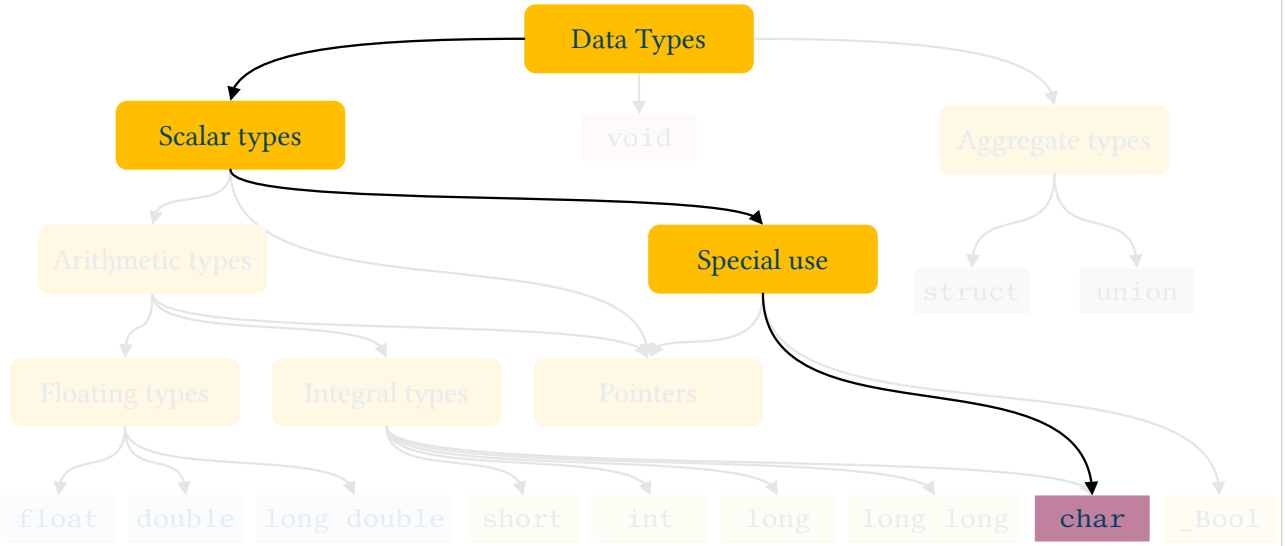
UNIVERSITÀ DI PARMA

- In the speed control a float was cast into an unsigned 16 bit integer
  - unsigned short
  - 0 – 65.535 range…
- ~370.000.000 USD

Ariane 5: 1996 disaster

Char Type

UNIVERSITÀ DI PARMA



Char Type

UNIVERSITÀ DI PARMA

- It is used to store numbers!
  - 0–255
  - -128–127
- But typically only used to store ASCII codes
- Format specifier
  - %c
- Constants:
  - Direct use of magic numb[illegible] 65
  - Use of correspondent symbol 'A'

The char type & scanf()

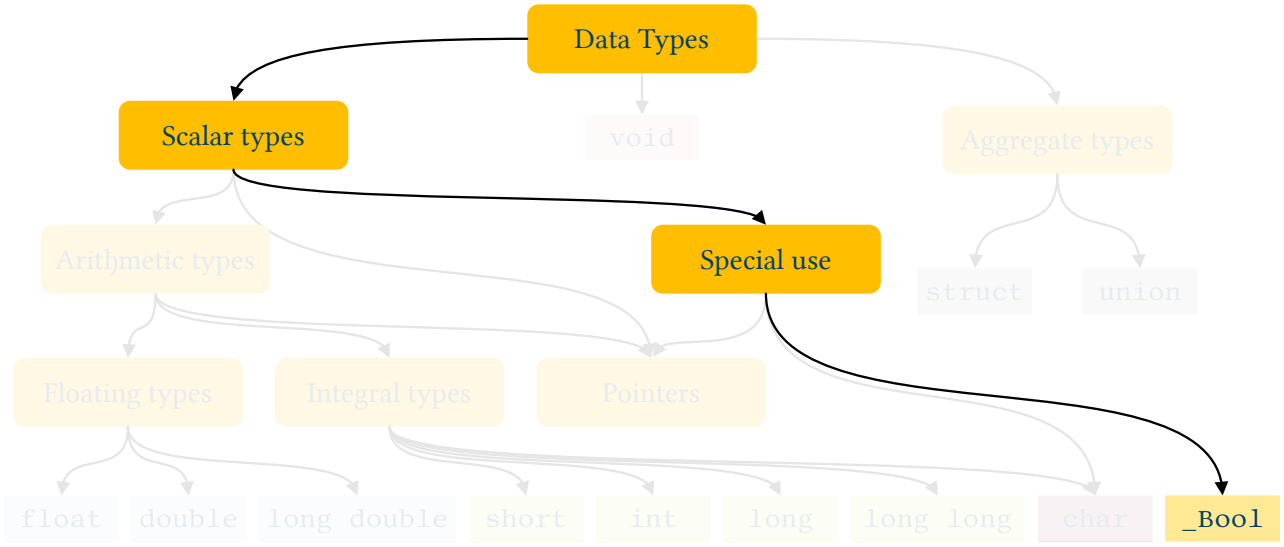
UNIVERSITÀ DI PARMA

- Reading the char and derived types using scanf() can be a little tricky
- When I type on the keyboard all characters are fed as input to the program
  - All means all. Namely, also the enter key…
  - All those "key presses" are initially stored in a input buffer


```
scanf("%c", &x); // I also read spaces and newlines

scanf(" %c", &x); // Ignore leading spaces or newlines
```

Boolean Type

UNIVERSITÀ DI PARMA



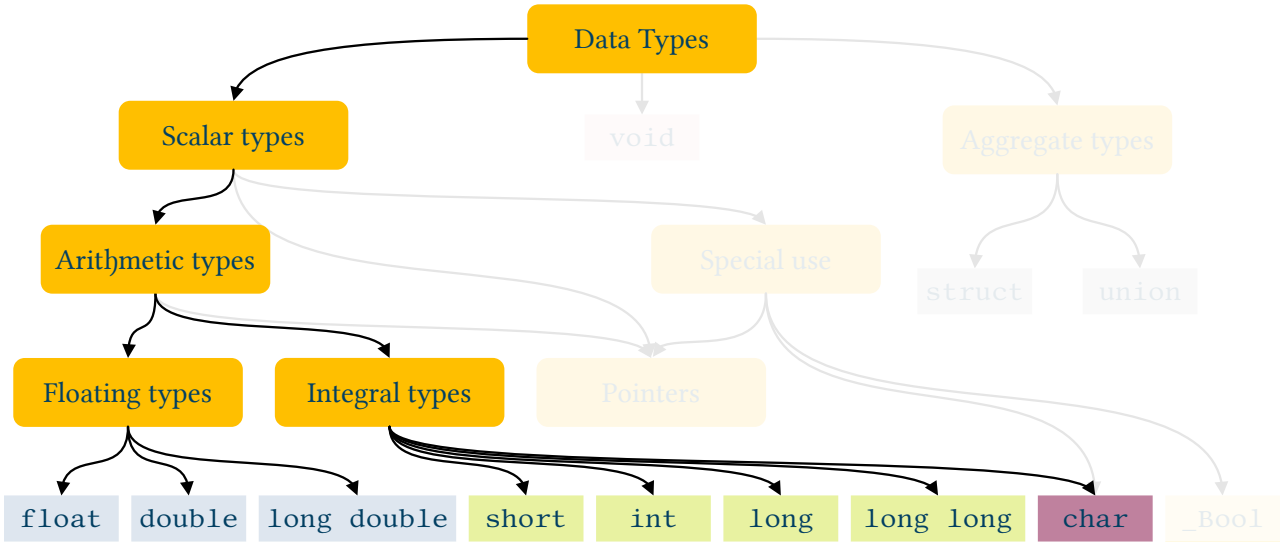
The _Bool Type

UNIVERSITÀ DI PARMA

- C99 (and later) specific
- A integer data type with the aim to only store “true” or “false”
  - 0 ⟶ false
  - 1 ⟶ true
- This type can then features 1 or 0 values only

“Arithmetic” issues

UNIVERSITÀ DI PARMA



Combining different data types

UNIVERSITÀ DI PARMA

- 3 floating point types and 8–10 integer ones
- Can I combine them in operations?
  - Yes
- What is the C behavior when I mix different kind of data types?
- Conversions
  - Assignment
  - Implicit or automatic conversions

Assignment conversions

UNIVERSITÀ DI PARMA

- When I operate an assignement (=)
  - The right operand (rvalue) is "casted" to the type I have on the left operand (lvalue)
- Issues
  - I can lost information — i.e.: float → int
  - Wrong result — i.e.: long long → short

Implicit or automatic casting

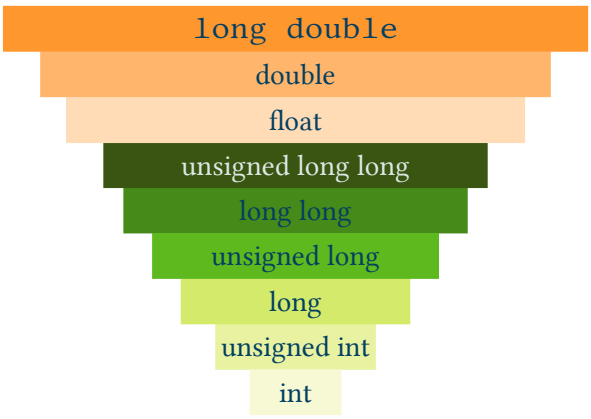
UNIVERSITÀ DI PARMA

- It occurs when we combine different types in expressions
- Binary operands needs same type operators
  - Arithmetic: *, /, %, +, -; Comparisons: <, >, <=, >=, ==, !=; Bitwise: &, ^, |,
- i.e.: a+b when a is an int and b is a float which kind of type gives as a result?
  - Spoiler: float
- Conversion rules
- High probability of bugs!

Implicit cast rules

UNIVERSITÀ DI PARMA

- Some types are converted to int or unsigned int
  - Even when we have same type operands
  - _Bool, char e short → int
    - Or unsigned int
- After this → conversion pyramid



Explicit cast

UNIVERSITÀ DI PARMA

- I select the conversion
  - This lead to avoid errors
- Syntax:

```
(type) expression
```

Variable scope

UNIVERSITÀ DI PARMA

- Can I use a variable everywhere in my code?
  - No!
- Visibility field or scope
  - Part of the code where a variable can be used
- 3 cases:
  - Program (global variables)
  - File
  - Block (local variables)

Global Variables

UNIVERSITÀ DI PARMA

- Defined outside of every code block
  - {}
- I can modify them everywhere
  - Data are NOT protected
  - Debug becomes more difficult
  - Uncertain
- Your final score will be, FOR SURE, affected
- One single benefit → initialization to 0

Local Variables

UNIVERSITÀ DI PARMA

- Defined
  - Inside a code block i.e. inside {}
  - In a for(;;)
  - As function arguments
- I can use them only inside the specif block where they are defined
- Automatic duration
  - Created when the execution flow meets that code block
  - Destroyed when that code block is over

Global vs Local

UNIVERSITÀ DI PARMA

- Local Variables
  - Let code blocks or functions to be self-sufficient
  - Easier to manage or to understand the code
    - Modularity
  - Less portability issues
- Global Variables
  - I have to track all the code parts where they are used
  - Require to coordinate team work
  - Memory is always used
  - Naming issues

Variables Duration

UNIVERSITÀ DI PARMA

- A variable is not always forever
- Static lifetime
  - Global Variables
    - Memory reserved when the execution begins and never freed
- Automatic lifetime
  - Local variables
    - Local variables are created when code execution reaches that block
    - And destroyed when the execution is over
    - Can I change this behavior? Yes → `static`

Constant values

UNIVERSITÀ DI PARMA

- We can need to use constant values
- Quick and dirt approach:
  - Put in my code a lot of "magic numbers"
  - This lead to a code really difficult to modify or maintain
- Good approach:
  - Define constant
  - Parametric code, very easy to modify or improve

$$e \quad \pi \quad \mu$$

How to define a constant value in C

UNIVERSITÀ DI PARMA

- Three different approaches
- #define
  - i.e.: `#define N_MAX_STUDENTS (100)`
- const prefix
  - I define and init a variable whose initial value can not be modified
  - In such a case the use of a global variable makes a sense
  - i.e.: `const int N_MAX_STUDENTS=100;`